Workforce Analytics — API Endpoint Documentation

Base URL (production): `https://activitymonitor.replit.app` All endpoints are under the `/api` prefix.

There are **two separate APIs** with **two separate authentication models**:

API	Who uses it	Auth
Agent / Sync	the desktop agent	per-device headers `x-device-id` + `x-device-secret`
Admin	the web dashboard	session cookie (login) + role `admin` / `super_user`

> There is **no public/unauthenticated data endpoint**. Every endpoint below either is the enrollment handshake (token-gated) or requires authentication.

Common conventions:

- Request/response bodies are JSON (`Content-Type: application/json`), except the screenshot upload, which is a direct binary `PUT` to object storage.
- Timestamps are ISO-8601 strings (e.g. `2026-06-02T14:30:00.000Z`).
- Errors return `{ "error": "message" }` with the HTTP status shown per route.

---

1. Agent / Sync API

Used by the desktop agent. Base path: `/api/sync` .

Auth headers (all routes except `/enroll`)

```
x-device-id:     <deviceId returned at enrollment>
x-device-secret: <deviceSecret returned ONCE at enrollment>
```

Missing/invalid credentials → `401` . A device whose consent was never recorded → `403` .

POST `/api/sync/enroll` — first-run registration (no auth header)

Requires a valid **enrollment token** (minted by an admin) **and** explicit consent. Returns the device id and a secret shown **exactly once**.

Request body

```
{
  "token": "string (enrollment token)",
  "hardwareHash": "string (stable per-machine fingerprint)",
  "systemName": "string (e.g. \"Jane's PC\")",
  "osType": "windows | macos | linux",
  "agentVersion": "string (optional)",
  "consentAcknowledged": true,
```

```
  "consentName": "string (name of the person who consented)"
}
```

`consentAcknowledged` **must** be the literal `true`. `consentName` is required.

Response `201`

```
{
  "deviceId": "uuid",
  "deviceSecret": "string (store securely; not retrievable again)",
  "config": {
    "monitoringEnabled": true,
    "screenshotMinMinutes": 5,
    "screenshotMaxMinutes": 15,
    "idleThresholdSeconds": 120,
    "syncIntervalSeconds": 60
  }
}
```

Errors: `400` invalid payload · `403` token invalid/expired/exhausted. Re-enrolling a known `hardwareHash` rotates its secret and refreshes consent.

POST `/api/sync/heartbeat` — liveness + config + commands

Request body

```
{ "agentVersion": "string (optional)" }
```

Response `200`

```
{
  "serverTime": "ISO-8601",
  "isLocked": false,
  "config": {
    "monitoringEnabled": true,
    "screenshotMinMinutes": 5,
    "screenshotMaxMinutes": 15,
    "idleThresholdSeconds": 120,
    "syncIntervalSeconds": 60
  },
  "commands": [
    { "id": "uuid", "commandType": "lock_screen | logout_user", "payload": null, "reason": "string|null" }
  ]
}
```

The agent should execute any returned commands and acknowledge them via `/commands/ack`.

POST `/api/sync/activity` — batch upload of activity logs

Request body (`logs`: 1–500 items)

```
{
  "logs": [
```

```
    {
      "processName": "string (required)",
      "windowTitle": "string (optional)",
      "startedAt": "ISO-8601 (required)",
      "endedAt": "ISO-8601 (required)",
      "durationSeconds": 0,
      "idleSeconds": 0
    }
  ]
}
```

Response `201`

```
{ "accepted": 1 }
```

Errors: `400` invalid payload (e.g. missing `endedAt`, empty or >500 logs). The server classifies each `processName` as productive / unproductive / neutral; unknown apps are auto-created as `undefined` for an admin to classify.

Screenshots — 3-step secure upload

Image bytes go **directly to object storage**, never through the API as base64.

Step 1 — POST `/api/sync/screenshots/request-url` (empty body)

```
{ "uploadURL": "https://storage.googleapis.com/...(presigned, ~15 min)",
  "storageKey": "/objects/uploads/<uuid>" }
```

Step 2 — `PUT <uploadURL>` — raw image bytes

```
PUT <uploadURL>
Content-Type: image/jpeg        (or image/png)
<binary image data>
```

A `2xx` response means the upload succeeded. (This request does **not** use the device auth headers — the URL itself is the short-lived credential.)

Step 3 — POST `/api/sync/screenshots` — record the metadata

```
{
  "storageKey": "/objects/uploads/<uuid>",   // echo the value from step 1
  "capturedAt": "ISO-8601",
  "fileSizeBytes": 12345
}
```

Response `201`

```
{ "id": "uuid" }
```

Errors: `400` if `storageKey` is not in the exact `/objects/uploads/<uuid>` shape the server issued in step 1.

POST `/api/sync/commands/ack` — report command progress

Request body

```
{ "commandId": "uuid", "status": "acknowledged | completed | failed" }
```

Response `200`

```
{ "id": "uuid", "status": "completed" }
```

Errors: `400` invalid body · `404` command not found for this device.

---

2. Admin API

Used by the web dashboard. Authentication is a **session cookie** obtained from login. All admin routes require the cookie **and** an `admin` / `super_user` role; writes (issuing commands, minting tokens, classifying apps) require `admin` / `super_user` specifically.

Authentication

POST `/api/auth/login`

```
{ "username": "string", "password": "string" }
```

→ `200` sets an httpOnly session cookie and returns `{ id, username, email, role, createdAt }`. Bad credentials → `401`.

POST `/api/auth/logout` → revokes the session, clears the cookie → `{ "ok": true }`.

GET `/api/auth/me` → current user `{ id, username, email, role, createdAt }` (`401` if not logged in).

Devices

Method & path	Description
`GET /api/devices`	List enrolled devices (with `online` flag).
`GET /api/devices/:id`	Device detail.
`GET /api/devices/:id/commands`	Command history (latest 50).
`POST /api/devices/:id/commands`	Issue a command (see below).

POST `/api/devices/:id/commands` body:

```
{ "commandType": "lock_screen | logout_user", "reason": "string (optional)" }
```

→ `201` returns the created command. `404` if device not found.

Activity

Method & path	Query params
`GET /api/activity`	`deviceId`, `userId`, `limit` (≤200)

Method & path	Query params
`GET /api/activity/timeline`	`deviceId` (latest 100)

Screenshots

Method & path	Description
`GET /api/screenshots`	List metadata; query `deviceId`, `limit` (≤200).
`GET /api/screenshots/:id/image`	Stream the image bytes (auth-gated).

List items include an `imageUrl` (`/api/screenshots/:id/image`) for display.

Reports

Method & path	Description
`GET /api/reports/summary`	KPIs: device counts, users, screenshots today, pending commands, today's productive/unproductive/neutral/undefined seconds.
`GET /api/reports/leaderboard`	Per-device productivity score for today.

Categories (app classification)

Method & path	Description
`GET /api/categories`	List app classification rules.
`PATCH /api/categories/:id`	Set `classification` (`productive` / `unproductive` / `neutral` / `undefined`) and/or `displayName`.

Enrollment tokens

Method & path	Description
`GET /api/tokens`	List enrollment tokens.
`POST /api/tokens`	Mint a token: `{ label?, maxUses? (1–1000), expiresDays? (1–365) }`. Returns the token row (incl. the token string).
`POST /api/tokens/:id/revoke`	Revoke a token.

Users

Method & path	Description
`GET /api/users`	List users (id, username, email, role, createdAt).

---

3. Health

GET `/api/healthz` (public) → `{ "status": "ok" }`.

Quick reference — typical agent lifecycle

1. **Enroll once:** `POST /api/sync/enroll` with a token + consent → save `deviceId` + `deviceSecret`.
2. **Every cycle:** `POST /api/sync/heartbeat` (apply config, run any commands), then `POST /api/sync/activity` with the latest log(s).
3. **Periodically:** request-url → `PUT` bytes → `POST /api/sync/screenshots`.
4. **On a command:** execute it, then `POST /api/sync/commands/ack`.